UNIT-2 AGILE MINI-PROJECT

Student Activities

1. Study the Business Problem and Product Vision

Students begin by reviewing the Product Discovery documents prepared in Unit 1 to establish a clear understanding of the software product before planning its development.

Activities include:

- Review the Product Vision Statement.
- Review the Mission Statement.
- Study the identified stakeholders.
- Analyze the Business Requirements.
- Review Functional Requirements.
- Review Non-functional Requirements.
- Study User Personas.
- Understand the proposed Minimum Viable Product (MVP).
- Review the Product Roadmap.

Students should ensure that they fully understand the objectives and scope of the Student Record and Academic Management Platform before translating it into Agile deliverables.

---

2. Define the Agile Product Vision

Students refine the Product Vision from an Agile perspective by identifying the overall purpose and expected outcomes of the project.

Activities include:

- Define the overall product objective.
- Identify the primary users of the system.
- Identify business benefits.
- Define measurable success criteria.

- Identify project constraints.
- Define expected software capabilities.

Example Product Vision:

"Develop a secure and user-friendly Student Record and Academic Management Platform that enables educational institutions to efficiently manage student records, attendance, academic performance, course registration, and reporting through a scalable web-based application."

Students document the Product Vision and align it with stakeholder expectations.

---

3. Identify Scrum Roles

Students organize themselves into Scrum Teams and assign project responsibilities.

Roles include:

Product Owner

Responsible for:

- Defining product requirements.
- Prioritizing the Product Backlog.
- Representing stakeholder interests.
- Accepting completed work.

Scrum Master

Responsible for:

- Facilitating Scrum ceremonies.
- Removing project impediments.
- Ensuring Agile practices are followed.
- Supporting team collaboration.

Development Team

Responsible for:

- Designing the application.

- Writing code.
- Testing software.
- Preparing documentation.
- Demonstrating completed features.

Students document the responsibilities of each role within their project team.

---

4. Identify Project Epics

Students organize the complete application into high-level functional modules known as Epics.

Example Epics include:

Epic 1 – Student Management

Features include:

- Student Registration
- Student Profile Management
- Student Search
- Student Update
- Student Deletion

Epic 2 – Course Management

Features include:

- Add Course
- Update Course
- Assign Students
- View Course Details

Epic 3 – Attendance Management

Features include:

- Record Attendance
- View Attendance
- Attendance Reports

Epic 4 – Marks and Academic Performance

Features include:

- Enter Marks
- Update Marks
- Calculate Grades
- Generate Result Sheets

Epic 5 – Authentication and Security

Features include:

- User Login
- Logout
- Password Reset
- Role-Based Access Control

Epic 6 – Notifications

Features include:

- Attendance Alerts
- Examination Notifications
- Result Notifications

Epic 7 – Reporting

Features include:

- Student Reports
- Attendance Reports
- Performance Reports
- Course Reports

Students create an Epic Document explaining the purpose and scope of each Epic.

---

5. Develop User Stories

Students break every Epic into multiple User Stories using the Agile User Story format.

Example:

User Story

As a Faculty Member, I want to record student attendance so that attendance records remain accurate and updated.

Acceptance Criteria:

- Faculty can select a course.
- Faculty can select a lecture date.
- Faculty can mark attendance.
- Attendance records are saved successfully.
- Attendance reports can be generated.

Students prepare User Stories for every identified feature.

---

6. Define Acceptance Criteria

For every User Story, students specify measurable conditions that determine whether the feature is complete.

Example:

User Story

Student Login

Acceptance Criteria:

- Login page loads successfully.
- Registered users can log in.
- Invalid credentials display appropriate error messages.
- Successful login redirects to the dashboard.
- Session management functions correctly.

Students understand that Acceptance Criteria provide a clear definition of "Done."

---

7. Prepare the Product Backlog

Students consolidate all User Stories into a prioritized Product Backlog.

Backlog prioritization is performed based on:

- Business value.
- User importance.
- Technical dependencies.
- Implementation complexity.

Example:

Priority	Epic	User Story
High	Authentication	User Login
High	Student Management	Add Student
High	Student Management	Search Student
High	Attendance	Record Attendance

Medium	Course Management	Add Course
Medium	Reporting	Attendance Report
Medium	Notifications	Attendance Alert
Low	Dashboard	Academic Analytics

Students learn how backlog prioritization supports incremental software delivery.

---

8. Estimate Story Points

Students estimate the effort required for each User Story using Story Points.

Example estimation scale:

Story Points	Complexity
1	Very Easy
2	Easy
3	Moderate
5	Difficult
8	Very Difficult
13	Highly Complex

Students participate in Planning Poker sessions to collaboratively estimate User Stories.

---

9. Sprint Planning

Students organize the Product Backlog into multiple development Sprints.

Example Sprint Plan:

Sprint 1

Goal:

Develop user authentication and project setup.

Activities:

- Repository initialization.
- User registration.
- Login functionality.
- Basic user interface.
- Testing.

Sprint 2

Goal:

Develop Student Management Module.

Activities:

- Add Student.
- Search Student.
- Update Student.
- Delete Student.

Sprint 3

Goal:

Develop Course and Attendance Management.

Activities:

- Add Course.
- Assign Courses.

- Record Attendance.
- Attendance Reports.

Students define Sprint Goals, Sprint Backlogs, and expected Sprint Deliverables.

---

10. Scrum Board Development

Students create a Scrum Board to monitor project progress.

Typical workflow:

```
----------------------------------------------------------

| Backlog | To Do | In Progress | Testing | Completed |

----------------------------------------------------------

| Login   | Add Student | Search Student | Reports | Setup |

| Course  | Attendance  | Notifications  |         |       |

----------------------------------------------------------
```

Students update the Scrum Board throughout Sprint execution.

---

11. Simulate Scrum Ceremonies

Each student team conducts simulated Scrum meetings.

Activities include:

Sprint Planning

- Select User Stories.
- Estimate effort.
- Define Sprint Goal.

Daily Scrum

Each member answers:

- What did I complete yesterday?

- What will I do today?
- What challenges am I facing?

Sprint Review

Teams demonstrate completed software features and collect feedback.

Sprint Retrospective

Teams discuss:

- What went well?
- What challenges occurred?
- What improvements should be implemented?

Students gain practical experience in Agile collaboration and communication.

---

12. Compare Agile Frameworks

Students prepare a comparative study of Agile frameworks introduced in the syllabus.

Comparison includes:

- Scrum
- Kanban
- Extreme Programming (XP)
- Lean Development
- Large Scale Scrum (LeSS)
- Scaled Agile Framework (SAFe)
- Scrum@Scale

Students recommend the most suitable framework for the Student Record and Academic Management Platform based on project size and complexity.

Student Deliverables

Upon completion of Unit 2, each student team submits:

- Agile Project Planning Report
- Product Vision Document
- Scrum Team Structure
- Scrum Roles and Responsibilities Document
- Epic Documentation
- User Story Repository
- Acceptance Criteria Document
- Prioritized Product Backlog
- Story Point Estimation Sheet
- Sprint Planning Document
- Sprint Backlog
- Scrum Board
- Burndown Chart
- Sprint Review Report
- Sprint Retrospective Report
- Agile Framework Comparison Report
- Project Timeline and Release Roadmap
- Complete Agile Documentation for the